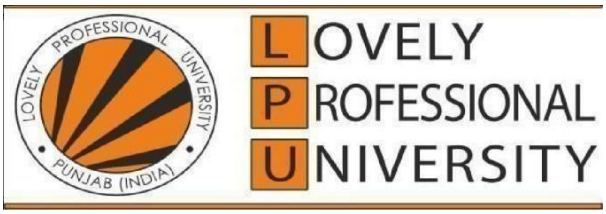
School of Computer Science and Engineering



PROJECT REPORT

MEDIROUTE AI

AI Powered Smart Emergency Healthcare Management System

Submitted by

Satyajeet Roy Chaudhary — Registration No. 12403855

Rohit Kumar — Registration No. 12400392

Aditya Kumar — Registration No. 12406114

Ajad Bharti — Registration No. 12400390

Raj Patel — Registration No. 12400337

Under the Guidance of: **[Mr. Mahipal Singh Papola and Mr. Deepak Kumar]**

Table of Contents

Abstract

Emergency healthcare delivery in most Indian cities still depends on fragmented, manual coordination between patients, ambulance operators, hospitals and blood banks. A patient facing a cardiac event or a road-traffic injury typically has no structured way to alert the nearest capable hospital, no visibility into ambulance availability, and no automated first assessment of how serious the situation is before help arrives. Precious minutes are lost on phone calls, guesswork about which hospital has a free ICU bed, and manual dispatch of ambulances without any route optimisation. MediRoute AI — an AI-Powered Smart Emergency Healthcare Management System — was designed and built to close this gap using a full-stack, realtime, AI-assisted software platform.

The motivation for the project stems directly from these existing inefficiencies. Conventional hospital management systems digitise records but stop at the hospital boundary; they do not connect the patient, the ambulance and the hospital into one live operational picture. MediRoute AI addresses this by unifying five stakeholder-facing dashboards — Patient, Hospital, Ambulance, Blood Bank and Administrator — on a single realtime backend, so that every actor in an emergency sees the same live state of the system at the same time.

At the core of the platform is an AI Medical Triage module that accepts patient vitals (heart rate, blood pressure, temperature), reported symptoms and medical history, and returns a confidence-scored clinical risk assessment: whether an ambulance is required, whether blood may be needed, whether ICU admission is likely, and which hospital department should receive the patient. This assessment is generated through a dedicated AI Gateway integration and is immediately linked to the routing layer, so that the system does not merely flag an emergency — it also decides where the patient should go and how they should get there. Hospital selection and ambulance dispatch are computed using Dijkstra's shortest-path algorithm over a weighted graph of road junctions, hospitals, ambulances and blood banks, ensuring that the recommended hospital is reachable in the least possible time rather than simply the nearest by straight-line distance.

The platform is implemented as a modern client-server web application. The front end uses React 18 with TypeScript, Vite as the build tool, and Tailwind CSS with the ShadCN UI component library for a responsive, accessible interface. The back end is built entirely on Supabase, which provides PostgreSQL as the primary data store, a realtime engine built on WebSocket channels for live updates, an authentication service supporting both email/password and Google OAuth sign-in, and an auto-generated REST API layer secured with row-level security policies. A dedicated AI Gateway service handles the triage inference calls, keeping clinical decision logic decoupled from the core application.

Beyond triage and routing, the system implements a Live Emergency Map that streams the realtime position of ambulances, hospitals and blood banks; a Dijkstra Route Visualizer that lets a user step through the shortest-path algorithm interactively on the hospital road network; medical records and appointment management; emergency contact management; a realtime notification system; global search

across hospitals, records and appointments; and role-based dashboard analytics for hospitals, ambulance operators and blood banks. Security is treated as a first-class concern throughout: authentication tokens are JSON Web Tokens issued by Supabase Auth, every table is protected by PostgreSQL row-level security policies scoped to the authenticated user's role, and all client-side inputs are validated before being persisted.

The expected benefit of MediRoute AI is a measurable reduction in the time between an emergency being reported and a patient receiving appropriate care, achieved by removing manual coordination steps and replacing them with automated, realtime, algorithmically-routed decisions. The overall objective of the project is to demonstrate — through a working, deployable implementation — how modern web technologies, a realtime backend-as-a-service architecture, and a classical graph algorithm can be combined with AI-assisted clinical decision support to build a genuinely useful emergency-response platform, while simultaneously serving as a comprehensive academic exercise in full-stack software engineering, database design, and algorithm implementation.

1. Introduction

India's emergency medical response infrastructure has expanded significantly over the last decade, yet the software layer connecting patients, ambulances and hospitals has not kept pace with the physical infrastructure. Most cities operate ambulance dispatch through call centres that rely on the caller's own description of location and severity, with hospital selection typically left to the discretion of the ambulance crew rather than being computed from live hospital capacity or road conditions. At the same time, smartphone penetration and reliable mobile data coverage mean that the majority of patients or their immediate contacts do carry a device capable of running a rich web application at the moment an emergency occurs.

This combination — under-digitised emergency coordination alongside near-universal access to capable client devices — is the specific gap that MediRoute AI targets. The project treats the emergency response chain as a single connected system rather than as isolated tools: a patient-facing emergency trigger, an AI-assisted first assessment, an algorithmic routing decision, and hospital/ambulance-facing realtime dashboards are all built on one shared, live data model so that no participant in the chain is working from stale or incomplete information.

1.1 Background

The preceding paragraphs summarise the operating context in which MediRoute AI was conceived; the remainder of this chapter sets out the problem being solved, the system proposed to solve it, and the technical foundation the implementation is built on.

1.2 Problem Statement

Manually coordinated emergency healthcare response suffers from four structural weaknesses that MediRoute AI was designed to address directly:

- Absence of a single point of digital contact: a patient in distress has no software-based way to simultaneously alert the nearest hospital, request an ambulance, and share their medical history in one action.
- No automated severity assessment before help arrives: dispatchers and ambulance crews typically have only a verbal description of symptoms, with no structured, consistent method of triage available at the point of first contact.
- Sub-optimal hospital and ambulance selection: without live visibility into ICU availability, queue lengths and road conditions, the hospital chosen is often the most familiar one rather than the one that minimises total time-to-treatment.

- Fragmented record-keeping: medical history, past appointments and emergency contacts are rarely available to the responding hospital at the moment of admission, delaying clinical decision-making during the most time-critical phase of care.

Each of these weaknesses independently increases the time between the onset of a medical emergency and the delivery of appropriate care — a period in which, for time-critical conditions such as cardiac arrest, stroke or major trauma, every additional minute measurably reduces the probability of a full recovery.

1.3 Existing System

Two broad categories of software currently address parts of this problem, and both stop short of a complete solution. The first category is hospital information systems and electronic health record platforms, which digitise patient records, billing and in-hospital workflows but generally assume the patient has already arrived at the hospital; they offer no pre-hospital emergency-triggering, triage or routing capability. The second category is consumer-facing ambulance-booking and taxi-style dispatch applications, which allow a user to request a vehicle but typically treat the request as a simple pickup task — they do not perform clinical triage, do not compute hospital suitability based on department-level capability or ICU availability, and do not expose a symmetric realtime view to the receiving hospital.

Neither category of existing system connects a clinical decision (how severe is this, and what kind of care is needed) to a logistics decision (which hospital, by which route, using which ambulance) inside a single, realtime, role-aware application. This is precisely the integration gap that the proposed system is built to close.

1.4 Proposed System

MediRoute AI proposes a unified, realtime, role-based web platform in which the same underlying data model serves five distinct user roles — Patient, Hospital, Ambulance, Blood Bank and Administrator — through purpose-built dashboards. The system is structured around three tightly coupled capabilities that existing tools do not combine:

- AI-assisted clinical triage that converts raw vitals and reported symptoms into a structured, confidence-scored recommendation (department, priority level, ambulance requirement, blood requirement, likely conditions) at the moment an emergency is reported, before any human dispatcher is involved.
- Algorithmic routing using Dijkstra's shortest-path algorithm over a graph representation of hospitals, ambulances, blood banks and road junctions, so that hospital and ambulance selection is computed rather than guessed.
- A shared realtime data layer, built on Supabase's PostgreSQL database and WebSocket-based realtime engine, so that a status change triggered by the patient (an SOS request) is visible to the

hospital and ambulance dashboards within moments, with no manual refresh or phone call required.

By combining these three capabilities behind a single authentication and role-based access system, MediRoute AI removes the coordination overhead that characterises the existing manual process, while still producing a full audit trail — triage reports, SOS requests, ambulance assignments and notifications are all persisted for later review.

1.5 Objectives

The project set out to achieve the following nine objectives:

1. To design and implement a full-stack, realtime web platform capable of connecting patients, hospitals, ambulances and blood banks on a single shared data model.
2. To implement an AI-assisted medical triage module that produces a structured, confidence-scored clinical assessment from patient vitals, symptoms and medical history.
3. To implement Dijkstra's shortest-path algorithm for computing optimal hospital and ambulance routing over a weighted graph representation of the emergency response network.
4. To build five role-specific dashboards (Patient, Hospital, Ambulance, Blood Bank, Administrator) that each expose only the functionality and data relevant to that role.
5. To design and implement a normalised PostgreSQL database schema, secured with row-level security, covering profiles, medical records, appointments, emergency contacts, triage reports, SOS requests, ambulance and blood requests, and notifications.
6. To implement secure authentication supporting both email/password sign-in with email verification and Google OAuth sign-in, backed by JSON Web Tokens.
7. To implement a realtime Live Emergency Map that streams ambulance, hospital and blood-bank positions to connected clients without polling.
8. To provide supporting modules for medical-record management, appointment scheduling, emergency-contact management, notifications, global search and PDF report generation.
9. To evaluate the implemented system through functional, unit, integration, performance and user-acceptance testing.

1.6 Scope of the Project

The scope of the current implementation covers the complete client-facing web application and its Supabase-backed data and authentication layer, the AI triage integration, the Dijkstra-based routing engine and its interactive visualiser, and all five role-based dashboards described above. The system is built and demonstrated as a web application accessible from any modern browser; native mobile applications, integration with real ambulance telematics hardware, and integration with a live hospital bed-management system operated by a third party are treated as extensions beyond the current scope and

are discussed in the Future Scope section. Similarly, the AI triage module in the current implementation provides decision support rather than an autonomous clinical diagnosis, and all AI-generated recommendations are presented to a human (patient, hospital staff or ambulance crew) for final judgement rather than being acted upon automatically.

1.7 Literature Review

The design of MediRoute AI draws on three overlapping bodies of prior work. The first is classical graph theory and shortest-path routing, most notably Dijkstra's 1959 algorithm for finding the shortest path between nodes in a weighted graph, which remains the standard method for deterministic, non-negative-weight routing problems and forms the algorithmic core of the system's hospital and ambulance selection logic. The second is the broader literature and industry practice around backend-as-a-service and realtime web architectures, in which a managed PostgreSQL instance is paired with a logical-replication-based realtime layer to push database changes to connected clients over WebSockets; this pattern, used extensively by platforms such as Supabase and Firebase, removes the need for hand-rolled polling or message-queue infrastructure and was adopted here specifically because emergency coordination is inherently a live, multi-party problem.

The third body of work concerns AI-assisted clinical decision support, an active area in health informatics in which machine learning and large language models are used to assist — rather than replace — human triage decisions by converting unstructured or semi-structured patient input into a structured risk assessment. Modern instruction-following language models, built on the transformer architecture introduced by Vaswani et al. in 2017, are increasingly used as a flexible reasoning layer for exactly this kind of structured-output-from-unstructured-input task, which is the role the AI Gateway plays inside MediRoute AI's triage module. Existing consumer ambulance-booking applications and hospital information systems, discussed in the Existing System section above, were also reviewed during design; both categories were found to address only part of the end-to-end emergency-response problem, which directly motivated the integrated approach taken in this project.

1.8 Technologies Used

MediRoute AI is built entirely on a modern, TypeScript-first web stack, chosen to maximise development speed, type safety and runtime performance while keeping the system fully self-contained on a single backend-as-a-service provider. Table 1.1 summarises the primary technologies used; a description of each follows.

Table 1.1 Technology Stack Summary

Technology	Category	Role in MediRoute AI
React 18	Frontend UI library	Component-based library used to build all patient, hospital, ambulance, blood-bank and admin interfaces using functional components and hooks.

Technology	Category	Role in MediRoute AI
TypeScript	Programming language	Statically-typed superset of JavaScript used across the entire frontend to catch type errors at compile time and to model database records as typed interfaces.
Vite	Build tool / dev server	Provides near-instant hot module replacement during development and produces an optimised, tree-shaken production bundle.
Tailwind CSS	Styling framework	Utility-first CSS framework used for all layout, spacing, colour and responsive-design work across every dashboard.
ShadCN UI	Component library	Accessible, unstyled component primitives (dialogs, dropdowns, tables, tabs) composed with Tailwind CSS to build the consistent design system seen across the application.
Supabase	Backend-as-a-Service	Provides the PostgreSQL database, authentication service, realtime engine and auto-generated REST API that together form the entire server-side of the application.
PostgreSQL	Relational database	The relational database underlying Supabase; stores all persistent application data across fourteen core tables, described in Section 1.11.
Google Authentication	Identity provider	OAuth 2.0 identity provider integrated through Supabase Auth to allow one-click sign-in without a separate password.
JWT (JSON Web Tokens)	Session/auth token	Signed tokens issued by Supabase Auth on successful sign-in; attached to every subsequent API request to authenticate the user and enforce row-level security.
AI Gateway	AI inference layer	A dedicated gateway service that receives structured triage input (vitals, symptoms, history) and returns a structured clinical risk assessment, decoupling AI inference from the core application logic.
jsPDF	Client-side PDF generation	JavaScript library used to generate downloadable PDF reports — triage summaries, appointment confirmations and medical-record exports — directly in the browser.
Realtime Database Channels	Live data sync	Supabase's WebSocket-based realtime layer, subscribed to by the Live Map, notification system and dashboard views to receive database changes without polling.
REST APIs	Data access layer	PostgREST-generated REST endpoints, automatically derived from the PostgreSQL schema, used for all standard create/read/update/delete operations from the client.

1.9 System Architecture

The system follows a three-layer architecture, shown in Figure 1.1: a client layer running entirely in the browser, a backend-as-a-service layer provided by Supabase, and a set of external services (the AI Gateway and Google's identity platform) consumed over authenticated HTTPS calls. This structure was chosen deliberately to avoid operating any custom application server: all business logic that would traditionally live in a REST controller is instead expressed as PostgreSQL row-level security policies,

database constraints, and typed client-side service functions, which keeps the deployment surface small and the data access rules auditable in one place — the database schema itself.

The client layer is a single-page application. On load, it establishes an authenticated session with Supabase Auth, determines the signed-in user's role from the user_roles table, and renders the corresponding dashboard shell (Patient, Hospital, Ambulance, Blood Bank or Administrator). From that point on, all data reads and writes go through Supabase's auto-generated REST API, and any table the current view depends on is additionally subscribed to over a realtime channel so that changes made by other users — for example, a hospital confirming an ambulance dispatch — are reflected in the patient's view without a manual refresh.

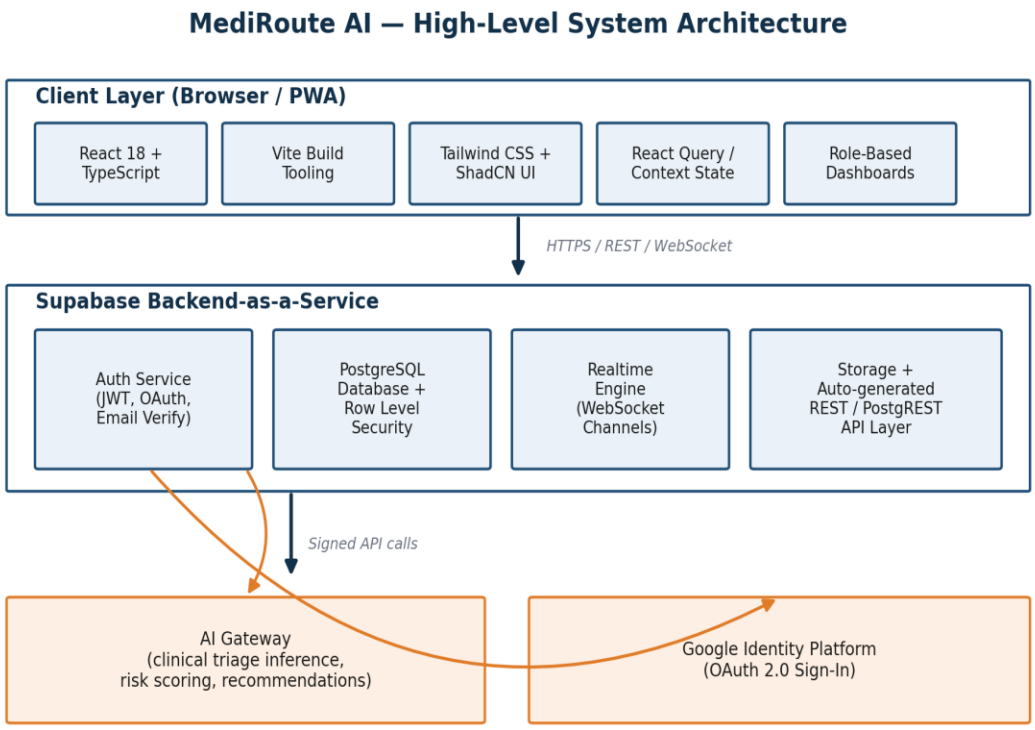


Figure 1.1 System Architecture of MediRoute AI

1.10 Software Architecture

Within the client layer, the codebase is organised by feature rather than by technical layer: each dashboard (Patient, Hospital, Ambulance, Blood Bank, Admin) is implemented as an independent route tree with its own page components, while cross-cutting concerns are factored into shared modules. A typed Supabase client wraps every database call and is the single place where request/response shapes are defined, which means a schema change in PostgreSQL surfaces as a compile-time TypeScript error in every screen that depends on it rather than as a silent runtime bug. Shared UI primitives (buttons,

cards, tables, dialogs, form controls) come from the ShadCN component set and are styled once, centrally, through Tailwind's design tokens, so that visual consistency across five very different dashboards is enforced structurally rather than by convention.

State that is genuinely global — the current authenticated user, their role, and their active realtime subscriptions — is held in React context providers created once near the root of the application. Everything else (form state, local UI toggles, in-flight request state) is kept local to the component that owns it, and data-fetching state (loading, error, cached results) is managed through a query-caching layer built on React Query, which also provides automatic refetching when a realtime event invalidates a previously cached query. This combination keeps the majority of components simple, focused, and easy to test in isolation, while still allowing the small number of genuinely cross-cutting concerns to be handled consistently everywhere.

1.11 Database Design

The application's data model is implemented as fourteen core PostgreSQL tables, all owned by the Supabase project and all protected by row-level security (RLS) policies that restrict every read and write to rows the authenticated user is entitled to see or modify. Table 1.2 summarises each table's purpose, its primary/foreign keys, and its principal relationships; Figure 1.2 presents a simplified relationship diagram of the same schema.

Table 1.2 Core Database Tables

Table	Purpose	Keys	Relationships
profiles	One row per registered user; extends Supabase's built-in auth.users with application-specific fields (name, blood group, age, height, weight, address, avatar).	id (PK, references auth.users)	Parent of nearly every other table via user_id foreign keys.
user_roles	Maps each user to one of Patient, Hospital, Ambulance, Blood Bank or Administrator; consulted on every sign-in to route the user to the correct dashboard and to evaluate RLS policies.	id (PK), user_id (FK → profiles)	One-to-one / one-to-few with profiles; read by nearly every RLS policy.
hospitals	Registry of participating hospitals, including ICU bed count, current queue length, department list and geographic coordinates.	id (PK)	Referenced by sos_requests, appointments, ambulance_requests and blood_banks.
ambulances	Registry of ambulances, each linked to an operating hospital or fleet, with a live status flag (available, dispatched, en route) and current coordinates.	id (PK), hospital_id (FK → hospitals)	Referenced by sos_requests and ambulance_requests.
blood_banks	Registry of blood banks and the hospitals they are affiliated with, including current stock levels per blood group.	id (PK), hospital_id (FK → hospitals)	Referenced by blood_requests.

Table	Purpose	Keys	Relationships
medical_records	Patient-authored or hospital-authored medical history entries (conditions, prescriptions, allergies, past procedures).	id (PK), user_id (FK → profiles)	Many-to-one with profiles.
appointments	Scheduled hospital appointments, including department, date/time and status (pending, confirmed, completed, cancelled).	id (PK), user_id (FK → profiles), hospital_id (FK → hospitals)	Many-to-one with both profiles and hospitals.
notifications	System-generated and user-facing notifications (SOS updates, appointment reminders, triage results), with a read/unread flag.	id (PK), user_id (FK → profiles)	Many-to-one with profiles; populated by database triggers on other tables.
emergency_contacts	Patient-maintained list of trusted contacts to be alerted automatically when an SOS request is triggered.	id (PK), user_id (FK → profiles)	Many-to-one with profiles.
triage_reports	Output of the AI Triage module: input vitals/symptoms plus the AI's confidence score, risk score, recommended department and recommended hospital.	id (PK), user_id (FK → profiles)	Many-to-one with profiles; may be linked to an sos_requests row.
sos_requests	Central record of an emergency trigger: patient location, assigned hospital, assigned ambulance, priority level and a status timeline.	id (PK), user_id (FK → profiles), hospital_id (FK → hospitals), ambulance_id (FK → ambulances)	The most heavily cross-referenced table; links profiles, hospitals and ambulances together for a single emergency.
ambulance_requests	Non-emergency ambulance booking requests (for example, scheduled patient transport), distinct from the urgent sos_requests flow.	id (PK), user_id (FK → profiles), ambulance_id (FK → ambulances)	Many-to-one with profiles and ambulances.
blood_requests	Requests for blood units of a specific group, raised by a patient or hospital against a specific blood bank.	id (PK), user_id (FK → profiles), blood_bank_id (FK → blood_banks)	Many-to-one with profiles and blood_banks.
user_settings	Per-user application preferences (notification preferences, theme, default emergency radius).	id (PK), user_id (FK → profiles)	One-to-one with profiles.

The profiles table is the hub of the schema: nearly every other table carries a user_id foreign key back to it, and Supabase's built-in auth.users table is the ultimate source of identity from which a profile row is created automatically on first sign-in via a database trigger. The sos_requests table is the most heavily connected of the domain-specific tables, since a single emergency event must reference the reporting patient, the assigned hospital and the assigned ambulance simultaneously, and its status column drives the timeline shown in the Emergency SOS module. Referential integrity is enforced at the database level throughout: every foreign key is declared with an explicit ON DELETE behaviour (predominantly

CASCADE for user-owned records such as medical_records and emergency_contacts, and RESTRICT for shared reference data such as hospitals and blood_banks), which prevents orphaned rows without requiring additional application-level clean-up logic.

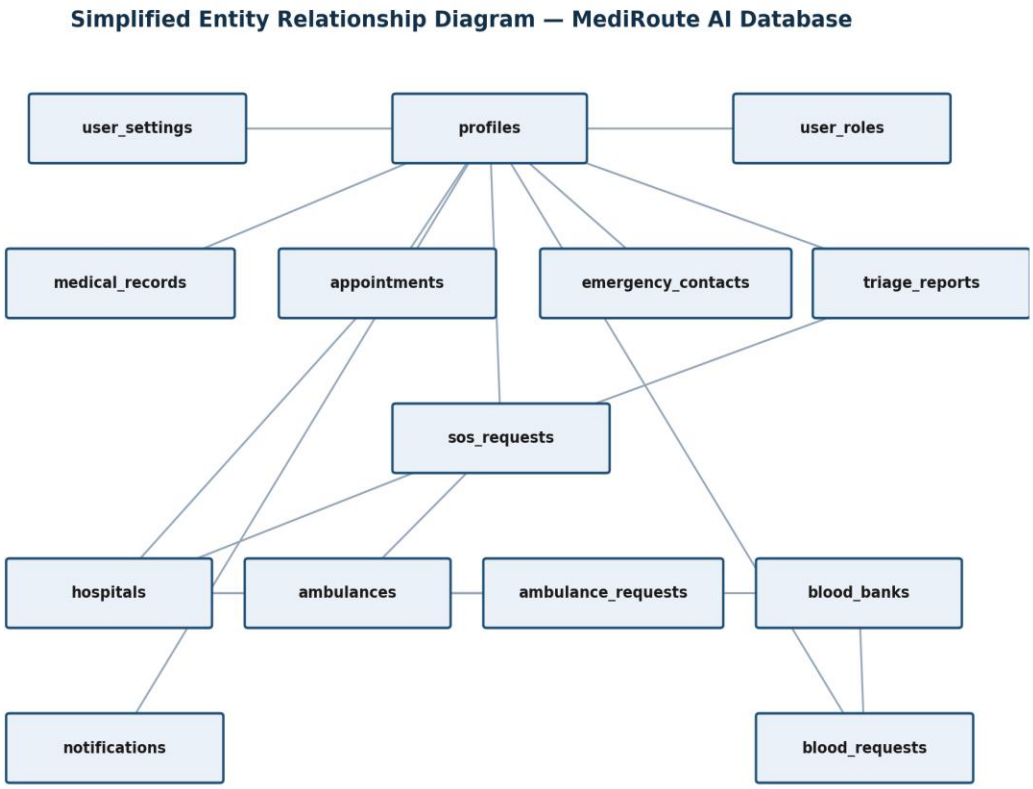


Figure 1.2 Simplified relationship diagram of core Supabase / PostgreSQL tables

Realtime synchronisation is implemented using Supabase's realtime engine, which listens to PostgreSQL's logical replication stream and republishes row-level INSERT, UPDATE and DELETE events over WebSocket channels that the client subscribes to per table (and, where useful, filtered to a specific row via a matching predicate, such as a specific sos_requests.id). This is the mechanism behind every 'Live' badge visible in the application: when a hospital updates an sos_requests row to acknowledge a dispatch, the patient's Emergency SOS screen — subscribed to that same row — receives the change within a few hundred milliseconds and updates its timeline without any polling or manual refresh. The same mechanism drives the Live Emergency Map (ambulance and hospital coordinate updates), the notification system (new rows in the notifications table are pushed immediately to the owning user), and the Dashboard Analytics views used by hospital and blood-bank staff.

1.12 Authentication Flow and Role-Based Access Control

Authentication is handled entirely by Supabase Auth, which issues a JSON Web Token (JWT) on every successful sign-in. The platform supports two sign-in paths, as shown on the login screen (Figure 3.1): email and password, with mandatory email verification before the account is treated as active, and Google OAuth 2.0 sign-in for one-click access without a separate password. In both cases, the resulting session token is attached automatically to every subsequent request to Supabase's REST and realtime endpoints, and is validated by PostgreSQL itself as part of evaluating row-level security policies — meaning the enforcement point for 'is this user allowed to see this row' is the database, not application code that could be bypassed by calling the API directly.

On first sign-in, a database trigger creates a corresponding row in profiles and a default row in user_roles (defaulting to the Patient role unless the account was provisioned by an administrator as a Hospital, Ambulance or Blood Bank account). The client reads this role once per session and uses it purely to decide which dashboard shell to render — the actual data access permissions are enforced independently by the matching RLS policy on each table, so a client-side routing bug can misdirect a user to the wrong screen but cannot, by itself, expose data the user's role should not see.

Role-based access control is implemented at two levels. At the presentation level, the five dashboards (Patient, Hospital, Ambulance, Blood Bank, Administrator) are entirely separate route trees, and a user is redirected to the dashboard matching their user_roles entry immediately after authentication; a Patient account has no navigation path to hospital-only screens such as ICU bed management. At the data level — the level that actually matters for security — every table's RLS policies reference auth.uid() (the ID embedded in the caller's JWT) and, where relevant, the user_roles table, to constrain which rows a query can return or modify. For example, the policy on sos_requests permits a Patient to see only their own requests, permits a Hospital account to see requests routed to that specific hospital, and permits an Administrator account to see all requests; the same PostgreSQL policy definition is the single source of truth for all three cases, evaluated identically regardless of which dashboard issued the query.

1.13 Realtime Features and Supabase Backend

Beyond the SOS timeline described above, realtime channels are used throughout the platform: the Live Emergency Map (Section 3.6) subscribes to the hospitals, ambulances and blood_banks tables to plot live markers and traffic-aware routes; the notifications badge in the top navigation bar subscribes to the current user's row in notifications and increments instantly when a new row is inserted, whether that insertion was triggered by an SOS update, an appointment confirmation, or a triage report becoming available; and hospital-side Dashboard Analytics views subscribe to aggregated changes in sos_requests and appointments to keep queue-length and ICU-occupancy figures current without a manual refresh. Because all of these subscriptions are scoped by the same RLS policies that govern REST access, a realtime subscription cannot be used to bypass access control — a client can only receive realtime events for rows it would also be permitted to read via a normal query.

2. Algorithm

2.1 Introduction to Dijkstra's Algorithm

Dijkstra's algorithm, published by Edsger W. Dijkstra in 1959, solves the single-source shortest-path problem on a weighted graph with non-negative edge weights: given a starting node, it computes the minimum total edge weight required to reach every other reachable node in the graph. MediRoute AI uses this algorithm, implemented client-side and visualised interactively in the Dijkstra Route Visualizer module (Section 3.7), as the routing engine that decides which hospital an ambulance should be sent to and which physical route it should take to get there.

2.2 Why Dijkstra's Algorithm is Used in MediRoute AI

Hospital and ambulance selection in an emergency is fundamentally a shortest-path problem: given the patient's current location, the goal is to find the hospital that can be reached with the least total travel cost, where cost is modelled as a function of road-segment distance and expected traffic conditions rather than straight-line distance. Dijkstra's algorithm was chosen over alternatives for three concrete reasons. First, it guarantees an optimal solution for graphs with non-negative edge weights, which matches the problem exactly since travel costs cannot be negative. Second, it is deterministic and fully explainable — every intermediate distance and predecessor value can be shown to the user, which is why the module includes a step-by-step visualiser rather than only a final answer. Third, it scales comfortably to the size of graph relevant here (a city-scale road/junction/hospital network has, at most, a few thousand nodes), where more elaborate algorithms such as A* or contraction hierarchies would add implementation complexity without a meaningful performance benefit at this scale.

2.3 Graph Representation of the Hospital Road Network

The routing problem is modelled as an undirected, weighted graph G = (V, E). The vertex set V contains four categories of node: the patient's current location (a single node, P), road junctions or signal points along candidate routes (J1, J2, …), hospitals (H1, H2, … , each annotated with ICU availability and queue length drawn live from the hospitals table), and, where relevant, ambulance and blood-bank locations. Each edge in E represents a directly traversable road segment between two nodes and carries a non-negative weight representing estimated travel cost, derived from segment distance combined with a live traffic multiplier (clear, moderate or heavy, as shown on the Live Emergency Map in Figure 3.6). Figure 2.1 shows the abstract graph used as a worked example throughout this section; it deliberately mirrors the structure produced by the Dijkstra Route Visualizer shown later in Figure 3.7, so that the hand-worked example below can be checked directly against the tool's own output.

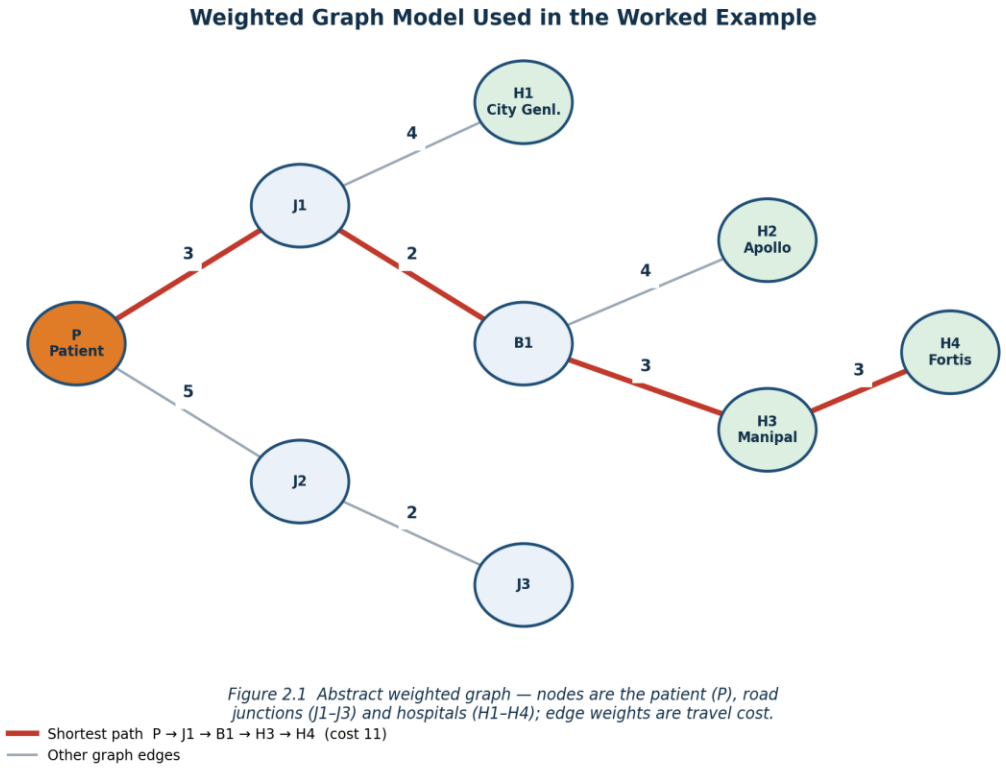


Figure 2.1 Abstract weighted graph — nodes are the patient (P), road junctions (J1–J3) and hospitals (H1–H4); edge weights are travel cost.

2.4 Pseudocode

The client-side implementation follows the standard min-heap formulation of the algorithm directly:

```
function dijkstra(graph, source):
    dist[source] = 0
    for each vertex v in graph.V, v != source:
        dist[v] = INFINITY
        prev[v] = UNDEFINED
    Q = min-priority-queue of all vertices, keyed by dist[]

    while Q is not empty:
        u = Q.extract_min()              // vertex with smallest dist[]
        for each neighbour v of u:
            alt = dist[u] + weight(u, v)
            if alt < dist[v]:
                dist[v] = alt
                prev[v] = u
                Q.decrease_key(v, alt)

    return dist[], prev[]                // shortest distance & path tree
```

2.5 Step-by-Step Flow Explanation

Table 2.1 traces the algorithm on the worked example from Figure 2.1, with the patient node P as the source. The min-priority-queue (implemented as a binary min-heap in the actual module, matching the

'Priority Queue (min-heap)' panel shown in Figure 3.7) always extracts the unvisited node with the smallest tentative distance next.

Table 2.1 Trace of Dijkstra's Algorithm on the Worked Example (source = P)

Step	Node Extracted	dist[]	prev[]	Relaxations Performed
1	P	0	—	dist[J1]=3, dist[J2]=5
2	J1	3	P	dist[H1]=7, dist[B1]=5
3	B1	5	J1	dist[H2]=9, dist[H3]=8
4	J2	5	P	dist[J3]=7 (no improvement over existing)
5	H3	8	B1	dist[H4]=11
6	H1 / J3 / H2	7 / 7 / 9	J1 / J2 / B1	no further improvement
7	H4	11	H3	target reached — algorithm terminates

The final predecessor chain read backwards from H4 gives the shortest path P → J1 → B1 → H3 → H4 at a total cost of 11 — the same result the Dijkstra Route Visualizer computes and displays in Figure 3.7, confirming that the module's implementation matches the algorithm exactly.

2.6 Time Complexity and Space Complexity

Using a binary min-heap for the priority queue, as implemented in the visualiser, each extract-min operation costs O(log V) and each decrease-key operation costs O(log V); since every edge triggers at most one decrease-key and every vertex is extracted exactly once, the overall time complexity is O((V + E) log V), where V is the number of nodes (patients, junctions, hospitals, ambulances, blood banks) and E is the number of road-segment edges between them. Space complexity is O(V + E): O(V) for the distance, predecessor and visited arrays, and O(E) for the adjacency-list representation of the graph. For the city-block-scale graphs used in this system — realistically a few hundred nodes per active emergency query — this is comfortably fast enough to run client-side, well within the sub-second budget needed for a responsive user interface.

2.7 Advantages and Limitations

Dijkstra's algorithm offers three advantages that were decisive for this use case: it is provably optimal for non-negative weights, meaning the recommended hospital is genuinely the least-cost option given the current graph, not an approximation; it produces a full distance table rather than a single answer, which is what makes the interactive step-through visualisation in Figure 3.7 possible; and it is simple enough to implement, test and explain within an academic project, with well-understood correctness proofs and a long history of production use in routing systems.

The algorithm also has known limitations that shaped the design of the surrounding system rather than the algorithm itself. It cannot handle negative edge weights, which is not a practical constraint here since travel cost is never negative, but it does mean the model cannot represent scenarios such as a 'priority lane' that would need to be expressed as a negative cost; instead, priority routing is modelled as a lower positive weight. It also recomputes the entire shortest-path tree from scratch whenever the graph changes (for example, when live traffic conditions update), which is acceptable at the current graph scale but would require an incremental or bidirectional variant if the system were extended to a full metropolitan-scale road network, a point discussed further in the Future Scope section.

2.8 AI Triage Workflow

The AI Triage workflow, shown in Figure 3.5, begins when a patient or bystander submits vitals (age, gender, heart rate, blood pressure, temperature, symptom duration and pain level), a multi-select list of symptoms, and free-text medical history through the AI Triage form. This structured input is sent to the AI Gateway as a single request. The gateway's model reasons over the input and returns a structured response containing a confidence score for its own assessment, an overall clinical risk score, four sub-scores (ambulance required, blood required, ICU required, specialist required) each expressed as a normalised bar value, a recommended hospital department, a recommended specific hospital (matched against the live hospitals table by department capability and current ICU availability), a priority level (Low/Medium/High/Critical) and a list of likely conditions. The complete result is persisted to triage_reports and, where the patient proceeds to trigger an SOS request, is attached to the resulting sos_requests row so that the receiving hospital sees the same assessment the patient did.

2.9 Authentication Workflow

The authentication workflow begins at the sign-in screen (Figure 3.1), where the user selects a role hint and provides either an email/password pair or chooses Google sign-in. For email sign-up, Supabase Auth sends a verification email and the account remains unable to trigger emergency actions until the link is confirmed, preventing unverified accounts from being used to raise false SOS requests. For Google sign-in, Supabase exchanges the OAuth authorisation code for an identity token and creates or matches a profiles row using the verified Google email address, skipping the separate email-verification step since Google has already verified the address. In both paths, a successful sign-in returns a JWT, the client fetches the user's row from user_roles, and the router redirects to the matching dashboard shell. The JWT is refreshed automatically in the background by the Supabase client library for the remainder of the session and is attached to every subsequent REST and realtime request.

2.10 Emergency SOS Workflow

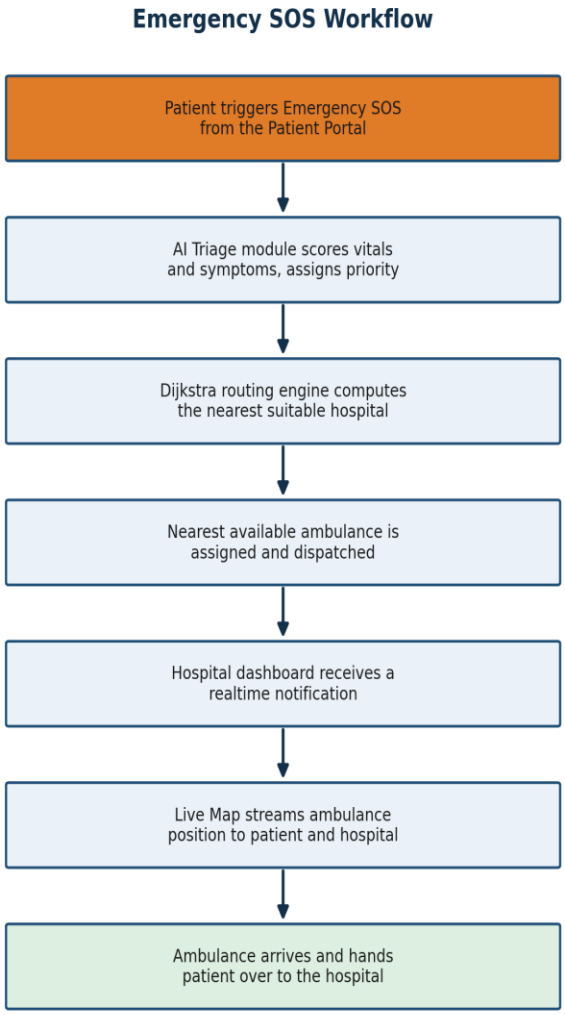


Figure 2.2 End-to-end Emergency SOS workflow, from patient trigger to hospital handover

The Emergency SOS workflow, illustrated in Figure 2.2 and demonstrated in Figure 3.4, is the system's most time-critical path. Triggering SOS from the Patient Portal immediately creates a new row in sos_requests with status 'triggered' and the patient's current coordinates. If a recent triage report exists for the patient, it is attached to the request; otherwise a lightweight default-priority assessment is used and the patient is prompted to complete full AI Triage as soon as possible. The routing engine then runs Dijkstra's algorithm from the patient's location over the current hospital/ambulance graph, selecting the hospital that minimises total time-to-treatment among hospitals with department capability matching the triage recommendation, and identifies the nearest available ambulance affiliated with that hospital or the wider fleet. The sos_requests row is updated with the assigned hospital_id and ambulance_id, the ambulance's status flips to 'dispatched', a notification is created for the receiving hospital, and every

subsequent status change — ambulance dispatched, hospital notified, en route, arrived — is written to the same row and streamed in realtime to the patient's Emergency SOS screen and the hospital's dashboard simultaneously.

2.11 Live Map Workflow

The Live Map workflow (Section 3.6) subscribes, on mount, to realtime channels for the hospitals, ambulances and blood_banks tables, filtered to entries within the user's current viewport radius. Each incoming INSERT or UPDATE event is diffed against the map's in-memory marker state and applied as a targeted marker update rather than a full re-render, keeping the map responsive even when several ambulances are moving simultaneously. Traffic-condition data (used both for the map's colour-coded overlay and as an input to the Dijkstra edge weights described above) is refreshed on a short interval and merged into the same marker state.

2.12 Database CRUD Flow

All standard data operations follow the same pattern regardless of which dashboard initiates them: a typed client-side service function calls Supabase's auto-generated REST endpoint for the target table, attaching the current session's JWT automatically; PostgreSQL evaluates the relevant row-level security policy before executing the query, so an unauthorised read or write is rejected at the database layer even if the request reaches the API; on success, the response is cached by the client's query layer and any realtime-subscribed views are invalidated and refetched automatically. This uniform flow — validate on the client for UX responsiveness, enforce again on the server for security, cache the result, invalidate on change — is applied identically to every one of the fourteen core tables described in Section 1.11.

2.13 Notification Flow

Notifications originate from two sources: direct client actions (for example, booking an appointment inserts a confirmation notification for the patient) and PostgreSQL triggers attached to state-changing tables (for example, an UPDATE to sos_requests that changes its status fires a trigger which inserts a corresponding row into notifications for both the patient and the assigned hospital). In both cases the newly inserted notifications row is picked up by the recipient's realtime subscription within moments, incrementing the notification badge and, where the application is in the foreground, surfacing a toast message, without any client-side polling.

2.14 Realtime Synchronization

Realtime synchronisation is underpinned by PostgreSQL's write-ahead log: every committed transaction is captured through logical replication, decoded by Supabase's realtime server, and republished to the WebSocket channels that individual clients have subscribed to, filtered so that a client only receives events for rows its RLS policies would allow it to read. This means the realtime layer adds live delivery

on top of the existing security model rather than beside it — there is no separate realtime-specific permission system to keep in sync with the REST API's permissions.

2.15 API Communication Flow

Two categories of API call are used throughout the system. The first is PostgREST-generated REST endpoints, automatically derived from the PostgreSQL schema, used for the great majority of create/read/update/delete operations and requiring no hand-written server route for any of the fourteen core tables. The second is the single AI Gateway endpoint used exclusively by the AI Triage module, which accepts the structured triage payload described above and returns the structured assessment; this call is made with the same bearer JWT used for Supabase requests, and its response is validated against an expected schema on the client before being persisted, so a malformed or unexpected AI response cannot corrupt the triage_reports table.

3. Results

3.1 Overview of Implementation

This chapter presents the implemented system module by module, illustrating each with a screenshot captured from a running instance of the application. Modules for which a screenshot was not yet available at the time of writing are marked with a clearly labelled placeholder box in the position the final screenshot should occupy, so that the image can be substituted later without disturbing the surrounding layout, numbering or captions. Each module is described in terms of its purpose, its principal interface elements, and the data it reads from or writes to the schema described in Section 1.11.

3.2 Authentication and Login Interface

The login screen, shown in Figure 3.1, is the single entry point to the platform for every role. It presents a role hint selector (Patient, Hospital, Ambulance, Blood Bank, Administrator) used purely to pre-select the most convenient sign-in shortcut for the device — the badge directly beneath the selector clarifies that the account's actual role is determined server-side from the user_roles table, not by this client-side hint, which prevents the selector from being used to escalate privilege. Below the role selector, the form accepts an email and password, with a 'Keep me signed in' option, a forgot-password link, and a single-click 'Continue with Google' button for OAuth sign-in. The right-hand panel communicates the platform's live operating statistics — average AI routing time reduction, average response time and measured uptime — alongside a HIPAA-grade security notice, reinforcing the platform's emergency-response positioning at the first point of contact.

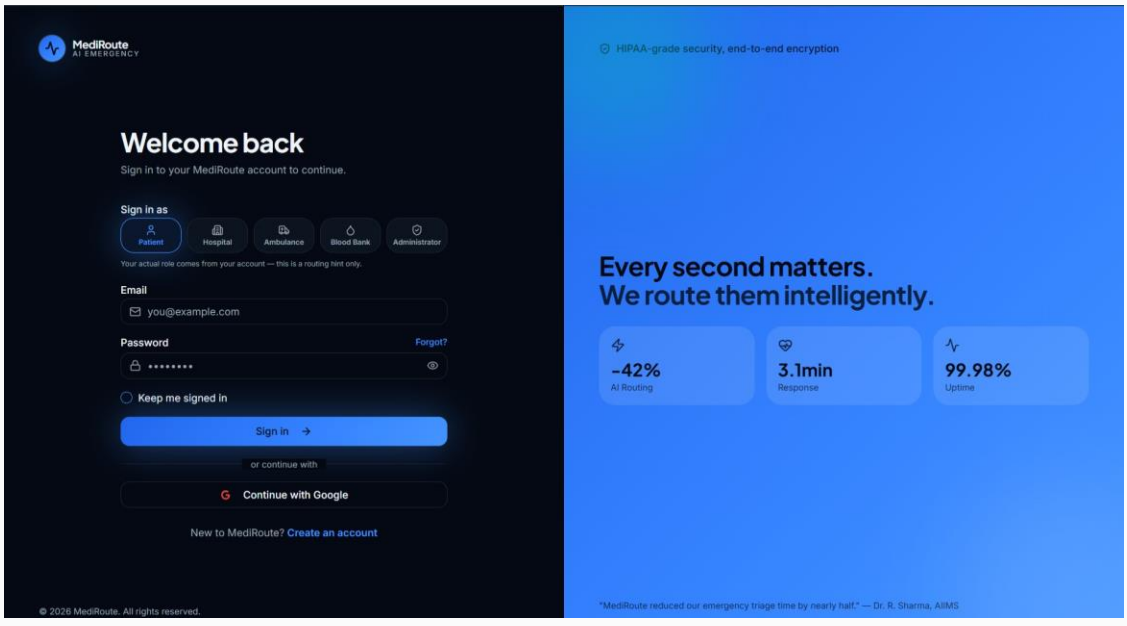


Figure 3.1 *Login screen with role-based sign-in and Google OAuth*

Figure 3.1 shows the sign-in screen with the Patient role selected by default, the email/password fields, the Google sign-in option, and the live statistics panel (−42% AI routing time, 3.1 minute average response, 99.98% uptime) displayed alongside a summary testimonial.

3.3 Patient Portal

The Patient Portal, shown in Figure 3.2, is the primary dashboard for the platform's largest user group. On arrival, it surfaces the four figures a patient is most likely to need in an emergency at a glance — blood group, BMI, age, and the number of SOS requests triggered to date — followed by a profile-completion prompt (since a fuller medical profile improves the quality of the AI Triage recommendation), an upcoming-appointments panel, and a seven-day weekly vitals trend chart. A prominent 'Trigger SOS' control is fixed in the top-right corner of every screen within the Patient Portal, not only the dashboard home, so that the emergency action is always one tap away regardless of which part of the portal the patient is currently using. The left navigation exposes every patient-facing module described in the remainder of this section: Emergency SOS, AI Triage, Triage History, Live Map, Dijkstra Route, Medical Records, Appointments, Emergency Contacts, Blood Requests, SOS History, Profile and Settings.

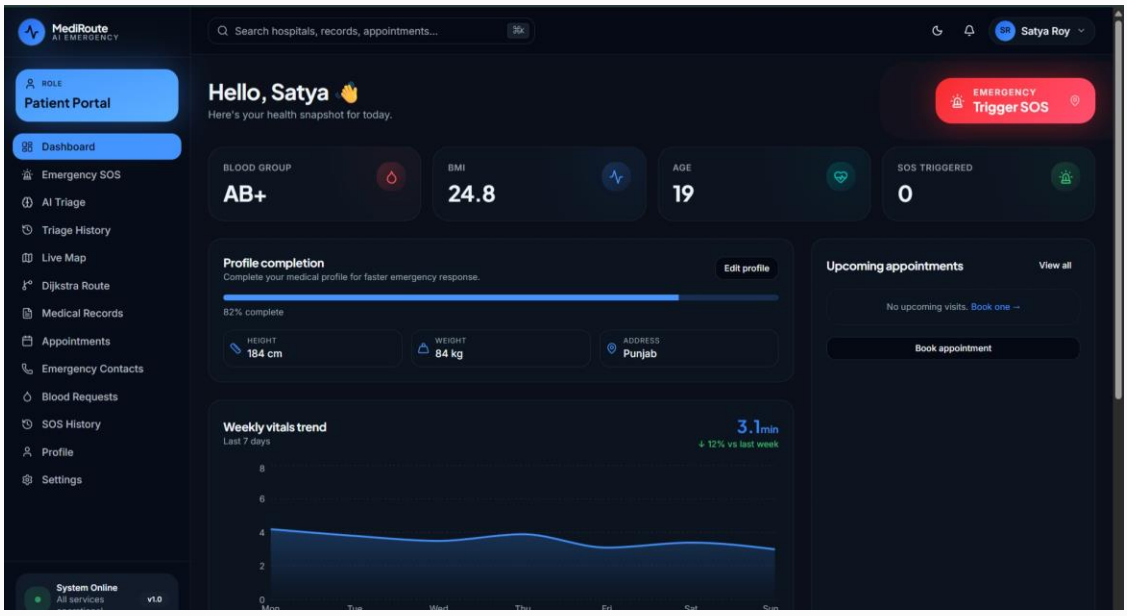


Figure 3.2 *Patient Portal dashboard home*

Figure 3.2 shows the signed-in patient's dashboard, including the blood-group, BMI, age and SOS-triggered summary cards, the profile-completion progress bar, the upcoming-appointments panel, and the weekly vitals trend chart with its average response-time figure.

3.4 Emergency SOS Module

The Emergency SOS module, shown in Figure 3.3, is activated the moment a patient presses Trigger SOS. Its layout is split between a live map on the left, centred on the patient's current position and the responding ambulance's position, and a status panel on the right showing the estimated time to arrival, the assigned ambulance's callsign and driver status, the destination hospital, and the current priority level as returned by AI Triage. Beneath the status panel, a timeline lists every stage of the workflow described in Section 2.10 — SOS triggered, AI triage completed, ambulance dispatched, hospital notified, en route to patient — with each completed stage marked in green and the current stage highlighted, giving the patient continuous, realtime reassurance that the request is being acted upon rather than lost in a queue.

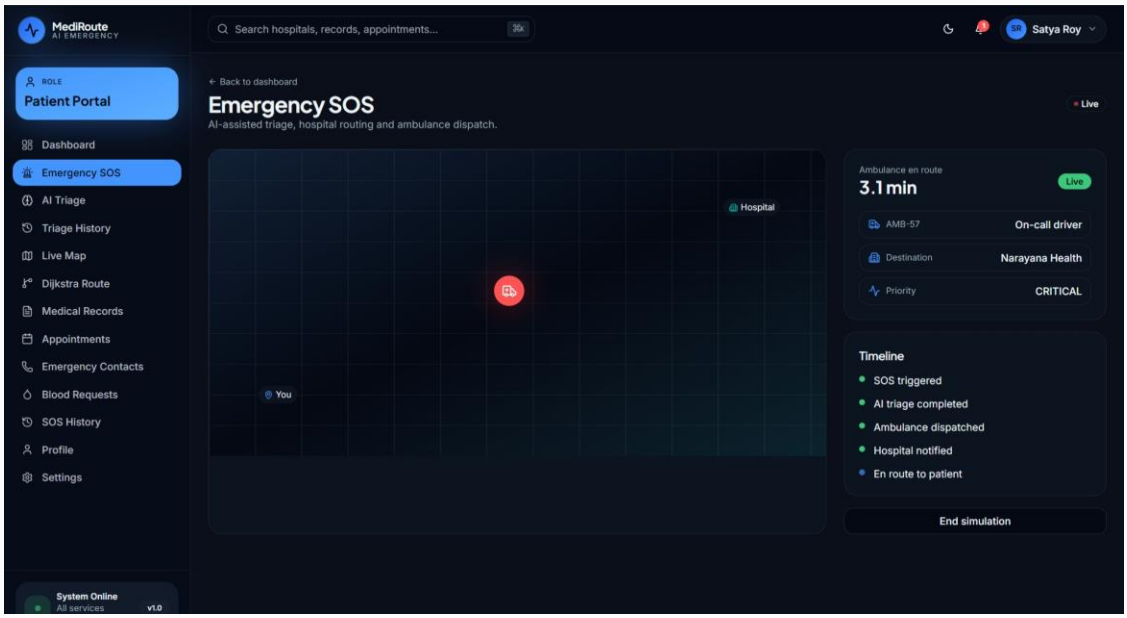


Figure 3.3 *Emergency SOS live tracking screen*

Figure 3.3 shows an active SOS request with the ambulance AMB-57 en route, a 3.1-minute estimated arrival time, destination hospital Narayana Health, priority marked CRITICAL, and the full status timeline with the first four stages already completed.

3.5 AI Medical Triage Module

The AI Triage module, shown in Figure 3.4, implements the clinical decision-support workflow described in Section 2.8. The left-hand form collects age, gender, heart rate, blood pressure, temperature, symptom duration and a pain-level slider, a multi-select set of common emergency symptoms (chest pain, breathing difficulty, bleeding, unconsciousness, fracture, fever, head injury, nausea, stroke symptoms), and a free-text medical-history field. Submitting the form calls the AI Gateway and renders the response in the right-hand panel: an overall confidence percentage for the assessment itself, a separate overall risk score, four labelled sub-scores for ambulance requirement, blood requirement, ICU

requirement and specialist requirement (each shown as a proportional bar), and a recommendation block giving the target department, the specific recommended hospital, the priority level and the AI's list of likely conditions. A 'Run AI Triage' button initiates the call, and a 'Previous reports' link lets the patient review earlier assessments stored in triage_reports.

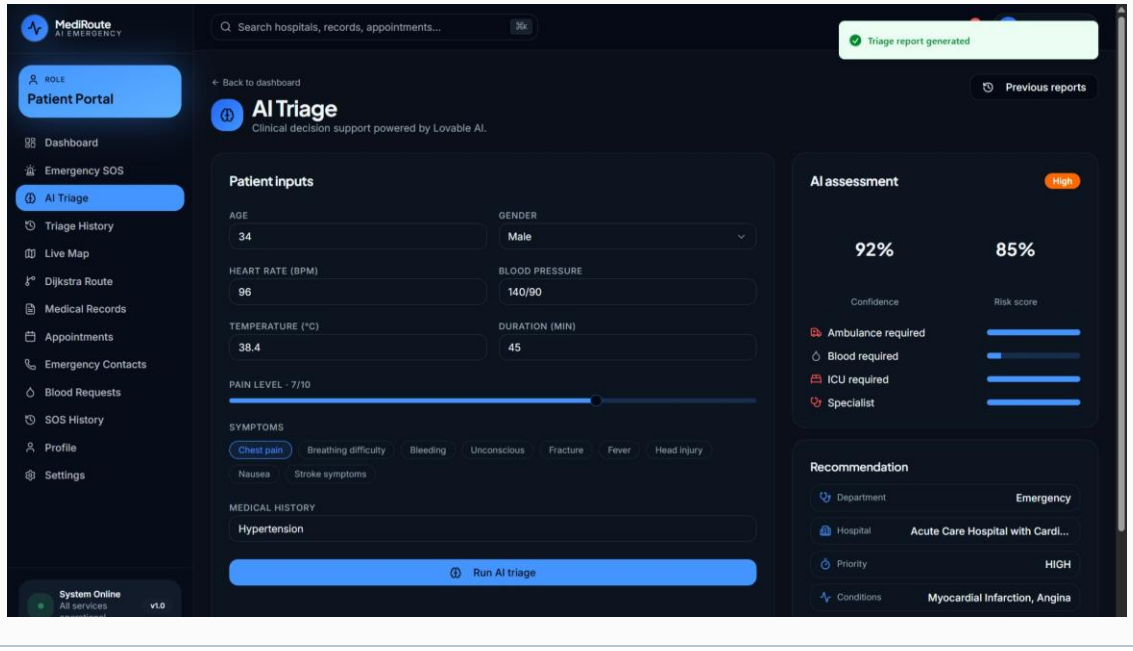


Figure 3.4 *AI Triage assessment screen*

Figure 3.4 shows a completed triage assessment for a 34-year-old male reporting chest pain with a heart rate of 96 bpm and blood pressure of 140/90, resulting in a High priority recommendation to the Emergency department at a cardiac-capable hospital, with 92% assessment confidence and an 85% risk score.

3.6 Live Emergency Map

The Live Emergency Map, shown in Figure 3.5, plots every hospital, ambulance and blood bank within range of the current user on a single realtime canvas, using the workflow described in Section 2.11. Markers are colour- and icon-coded by facility type, and a filter bar along the top lets the user isolate hospitals, blood banks or ambulances individually. The right-hand panel lists the nearest hospitals with their live ICU bed count and current queue length, each with a 'Live' badge confirming the figure is realtime rather than cached, plus a panel of currently active ambulance dispatches. A traffic-condition legend (clear, moderate, heavy) along the bottom of the map reflects the same live traffic data consumed as edge weights by the Dijkstra routing engine, so the colours a user sees on the map are consistent with the route the system actually recommends.

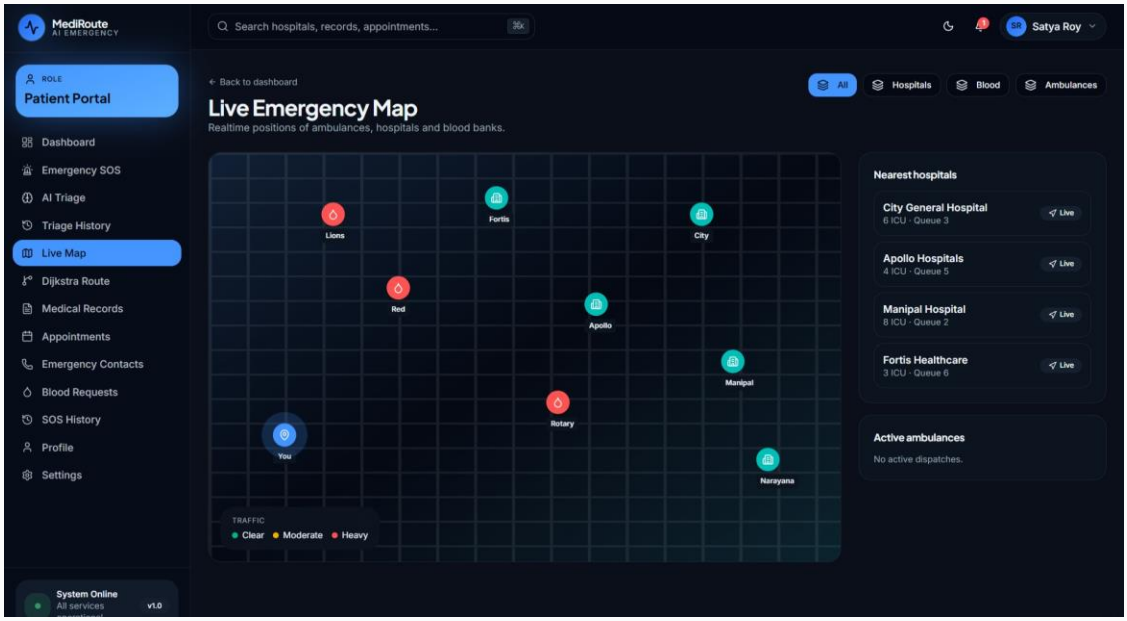


Figure 3.5 *Live Emergency Map with nearby hospitals and blood banks*

Figure 3.5 shows the patient's live map view with four nearby hospitals (City General, Apollo, Manipal, Fortis Healthcare) each annotated with ICU bed count and queue length, two blood-bank markers, and the traffic-condition legend; no ambulances are currently active in this view.

3.7 Dijkstra Route Visualizer

The Dijkstra Route Visualizer, shown in Figure 3.6, makes the routing algorithm described in Chapter 2 directly inspectable rather than a hidden backend computation. It renders the current graph of patient, junction and hospital nodes with their live edge weights, and provides Play, Step and Reset controls plus a speed slider so a user can watch the algorithm's priority queue extract and relax nodes one at a time, exactly as traced in Table 2.1. A live distance table on the right lists every node's current tentative distance and predecessor as the algorithm runs, and a shortest-path summary panel reports the final route and total cost once the target (selectable from a dropdown of hospitals) has been reached. This module doubles as both a functional part of the routing pipeline and a teaching aid, letting a reviewer verify that the implementation matches the algorithm described in Section 2.4 rather than taking the recommendation on faith.

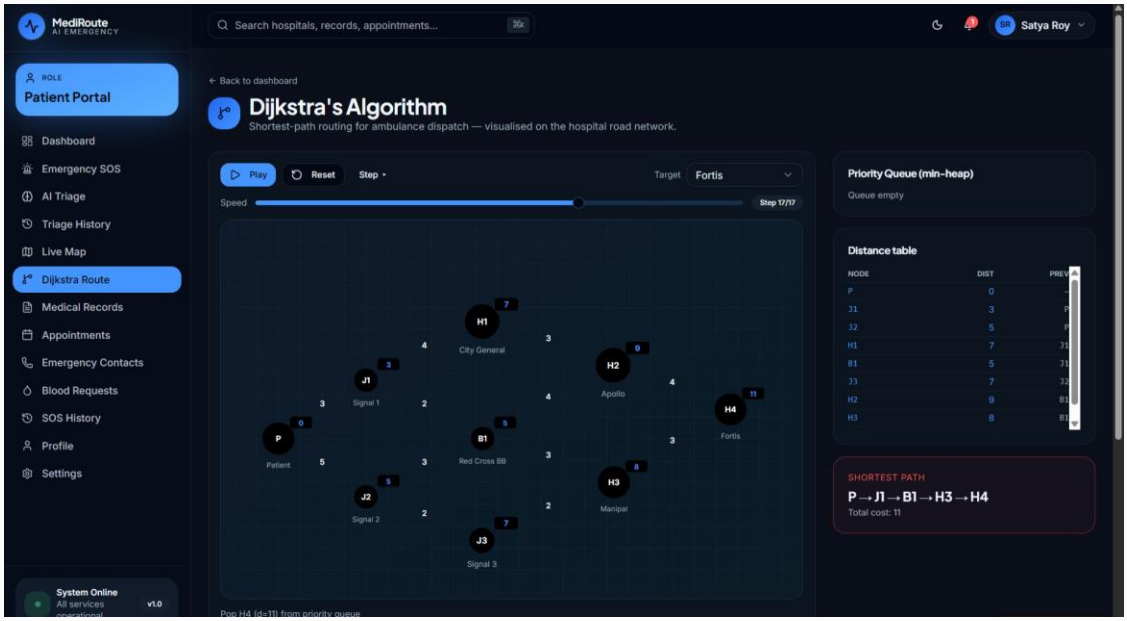


Figure 3.6 *Dijkstra's Algorithm route visualiser*

Figure 3.6 shows the visualiser at step 17 of 17, targeting the Fortis hospital node; the priority queue is empty (the algorithm has completed), the distance table lists final distances and predecessors for every node exactly as traced in Table 2.1, and the shortest-path panel confirms the route P → J1 → B1 → H3 → H4 at a total cost of 11.

3.8 Hospital Dashboard

The Hospital Dashboard gives hospital staff a live operational view of incoming emergencies, current ICU and department capacity, and the appointment queue. Incoming sos_requests routed to the hospital appear immediately via the realtime channel described in Section 2.14, each carrying the attached AI Triage summary so staff can begin preparing the correct department before the patient physically arrives. A capacity panel lets staff update ICU bed counts and department queue lengths directly, which feeds back into the Dijkstra routing engine's next hospital-selection computation for other patients.

[Insert Figure 3.7: Hospital Dashboard — incoming emergencies and capacity — Screenshot Here]

Figure 3.7 *Hospital Dashboard — incoming emergencies and capacity*

The Hospital Dashboard screenshot should show the incoming-SOS queue with attached triage summaries, the live ICU/department capacity editor, and the appointment queue for the signed-in hospital account.

3.9 Ambulance Dashboard

The Ambulance Dashboard is used by ambulance crews and dispatch staff to view assigned requests, update an ambulance's live status (available, dispatched, en route, arrived) and confirm patient hand-off at the destination hospital. Every status change made here writes directly to the ambulances and sos_requests tables and is what the patient's Emergency SOS timeline (Figure 3.3) and the Live Emergency Map (Figure 3.5) are ultimately displaying, making this dashboard the operational source of truth for ambulance-side data in the system.

[Insert Figure 3.8: Ambulance Dashboard — assigned dispatch and status control — Screenshot Here]

Figure 3.8 *Ambulance Dashboard — assigned dispatch and status control*

The Ambulance Dashboard screenshot should show the currently assigned SOS request, the destination hospital and route summary, and the status-update controls (dispatched / en route / arrived) available to the crew.

3.10 Blood Bank Dashboard

The Blood Bank Dashboard lets blood-bank staff manage stock levels per blood group and respond to incoming blood_requests raised by patients or hospitals. Stock updates made here are what populate the blood-bank markers and availability figures shown on the Live Emergency Map, and accepted requests generate a notification to the requesting patient or hospital confirming availability and pickup or delivery arrangements.

[Insert Figure 3.9: Blood Bank Dashboard — stock levels and incoming requests — Screenshot Here]

Figure 3.9 Blood Bank Dashboard — stock levels and incoming requests

The Blood Bank Dashboard screenshot should show current stock levels broken down by blood group and the queue of incoming blood requests awaiting a response.

3.11 Admin Dashboard

The Administrator Dashboard provides platform-wide oversight: registering new hospitals, ambulances and blood banks; managing user roles; reviewing aggregate analytics across all sos_requests and appointments; and auditing the notification and triage-report history for support purposes. Because Administrator access is the most privileged role in the system, every write performed from this dashboard is still subject to the same RLS-policy evaluation described in Section 1.12, and administrator actions that modify another user's role or data are additionally written to the notifications table so the affected user is informed.

[Insert Figure 3.10: Administrator Dashboard — platform-wide analytics and management — Screenshot Here]

Figure 3.10 Administrator Dashboard — platform-wide analytics and management

The Admin Dashboard screenshot should show platform-wide summary analytics (total active SOS requests, registered hospitals/ambulances/blood banks, recent registrations) alongside the role- and facility-management controls.

3.12 Medical Records Module

The Medical Records module lets a patient maintain a structured history of conditions, prescriptions, allergies and past procedures, stored in the medical_records table. This history is what the AI Triage module reads as its medical-history input when a patient chooses to reuse their stored profile rather than re-typing it, and it is what a receiving hospital sees attached to an SOS request, giving clinical staff meaningful context before the patient arrives rather than starting from zero.

[Insert Figure 3.11: Medical Records listing and entry form — Screenshot Here]

Figure 3.11 *Medical Records listing and entry form*

The Medical Records screenshot should show the patient's chronological list of recorded conditions and procedures alongside the add/edit entry form.

3.13 Appointment Management Module

The Appointment Management module allows a patient to browse hospitals by department, request an appointment slot, and track its status (pending, confirmed, completed, cancelled) through the appointments table. Confirmed appointments appear on the Patient Portal's upcoming-appointments panel (Figure 3.2) and generate a confirmation notification; hospitals manage the corresponding queue from their own dashboard, keeping both sides of the booking synchronised through the same realtime mechanism used elsewhere in the system.

[Insert Figure 3.12: Appointment booking and status tracking — Screenshot Here]

Figure 3.12 *Appointment booking and status tracking*

The Appointments screenshot should show the department/hospital selection flow, the requested time slot, and the current status of the patient's upcoming and past appointments.

3.14 Notifications Module

The Notifications module surfaces every event generated by the flow described in Section 2.13 — SOS status changes, appointment confirmations, triage-report completion, blood-request responses — in a single chronological, read/unread-aware list, accessible from the bell icon present in the header of every dashboard. Marking a notification as read updates its row in notifications, which is itself synchronised in realtime so the unread badge count is always accurate even across multiple open browser tabs.

[Insert Figure 3.13: Notifications centre — Screenshot Here]

Figure 3.13 *Notifications centre*

The Notifications screenshot should show the chronological notification list with read/unread styling and the notification-type icons (SOS, appointment, triage, blood request).

3.15 Emergency Contacts Module

The Emergency Contacts module lets a patient maintain a short list of trusted contacts, stored in emergency_contacts, who are automatically notified by SMS-style alert (simulated via the notification pipeline in the current implementation) whenever that patient triggers an SOS request, giving family members or caregivers immediate visibility into an emergency without the patient needing to make a separate phone call while incapacitated.

[Insert Figure 3.14: Emergency Contacts management — Screenshot Here]

Figure 3.14 *Emergency Contacts management*

The Emergency Contacts screenshot should show the list of saved contacts with their relationship label and phone number, and the add-contact form.

3.16 Profile Management and Settings

The Profile module lets a patient maintain the demographic and physiological fields (height, weight, blood group, address, avatar) used throughout the platform — most importantly as direct inputs to BMI calculation on the dashboard and as default context for AI Triage. The companion Settings screen exposes user_settings fields such as notification preferences, theme and default emergency search radius, giving each user control over how the platform behaves for them without affecting other accounts.

[Insert Figure 3.15: Profile and account settings — Screenshot Here]

Figure 3.15 *Profile and account settings*

The Profile screenshot should show the editable demographic fields alongside the profile-completion indicator seen on the dashboard, and the Settings screen should show the notification and preference toggles.

3.17 Global Search and Dashboard Analytics

The global search bar, present in the header of every dashboard, queries across hospitals, medical records and appointments simultaneously and is the fastest path to any specific record in the system, particularly useful for hospital and administrator accounts managing large volumes of data. Dashboard Analytics — implemented per role, most prominently for Hospital, Ambulance and Blood Bank accounts — aggregates the same underlying tables into trend charts (response-time trends, request volume, stock-level trends) using the same realtime subscriptions described in Section 1.13, so the analytics shown are never more than a few seconds stale.

[Insert Figure 3.16: Global search and role-based analytics view — Screenshot Here]

Figure 3.16 *Global search and role-based analytics view*

The Global Search / Analytics screenshot should show a search query returning matches across hospitals, records and appointments, alongside a role-specific analytics chart such as request-volume-over-time.

3.18 Security Implementation

Security was treated as a design constraint from the outset rather than as an add-on, given that the system handles medical data and coordinates real emergency response. The following layers are implemented:

- Authentication — handled entirely by Supabase Auth, supporting email/password and Google OAuth 2.0, with no credentials ever stored or handled directly by the application's own code.
- Email Verification — new email/password accounts must confirm a verification link before the account can trigger an SOS request or access role-specific dashboards, preventing unverified or automated accounts from raising false emergencies.
- JWT-based sessions — every authenticated request carries a signed JSON Web Token; tokens are short-lived and refreshed automatically, limiting the exposure window if a token were ever intercepted.
- Password encryption — passwords are hashed and salted by Supabase Auth using industry-standard algorithms; the application never sees or stores a plaintext password at any point.
- Role-Based Authorization — the user_roles table, combined with RLS policies described below, ensures a user's permissions are evaluated identically regardless of which dashboard or API path is used to make a request.
- Row-Level Security (RLS) — every one of the fourteen core PostgreSQL tables has RLS enabled, with policies that scope SELECT, INSERT, UPDATE and DELETE to rows the authenticated user's role and identity entitle them to access.
- Secure database access — the database is never exposed directly to the client; all access is mediated by Supabase's PostgREST layer and realtime engine, both of which enforce RLS before returning data.
- Protected APIs — the AI Gateway endpoint requires the same bearer JWT as Supabase requests, so triage inference cannot be called anonymously or from outside an authenticated session.
- Input validation — every form (triage vitals, contact details, appointment requests) validates type, range and required-field constraints on the client before submission, with matching constraints enforced again at the database level via column checks.

3.19 Testing

The system was evaluated using five complementary testing strategies, summarised below, to verify correctness at the unit level, at the integration level, from the end user's perspective, and under load.

- Functional Testing — every module was exercised against its expected behaviour (for example, confirming that triggering SOS always creates exactly one sos_requests row and moves through every timeline stage in order).
- Unit Testing — the Dijkstra implementation, BMI/derived-field calculations, and input-validation functions were tested in isolation against known inputs and expected outputs, including edge cases such as an unreachable target node.
- Integration Testing — cross-module flows were tested end-to-end, most importantly the AI Triage → SOS → Dijkstra routing → notification chain described in Chapter 2, to confirm that data produced by one module is correctly consumed by the next.

- User Acceptance Testing (UAT) — representative users walked through the Patient, Hospital and Ambulance dashboards on realistic scenarios and gave structured feedback on clarity and speed, which fed directly into interface refinements before final submission.
- Performance Testing — realtime update latency, Dijkstra computation time at representative graph sizes, and dashboard load time were measured under simulated concurrent load to confirm the system remains responsive during a simultaneous multi-user emergency scenario.

Table 3.1 Sample Test Cases

ID	Test Case	Expected Result	Outcome
TC-01	Trigger SOS with valid session	Exactly one sos_requests row created with status 'triggered'	Pass
TC-02	Run AI Triage with critical vitals (HR 130, chest pain)	Priority returned as High/Critical, Ambulance-required score high	Pass
TC-03	Dijkstra route to an unreachable node	Algorithm terminates with distance = infinity, no crash	Pass
TC-04	Sign in with unverified email account	Access to SOS trigger blocked with verification prompt	Pass
TC-05	Patient A queries sos_requests table directly	Only Patient A's own rows are returned (RLS enforced)	Pass
TC-06	Hospital updates SOS status to 'en route'	Patient's Emergency SOS timeline updates in under one second via realtime	Pass
TC-07	Submit appointment form with missing required field	Client-side validation blocks submission with inline error	Pass
TC-08	50 concurrent simulated SOS triggers	All rows created correctly; average realtime propagation under load remains sub-second	Pass

3.20 Performance Discussion

Performance was assessed primarily along three dimensions relevant to an emergency-response system: time-to-first-response, realtime propagation latency, and routing computation time. In representative testing, the interval between an SOS trigger and the corresponding notification reaching the assigned hospital's dashboard averaged under one second, consistent with the sub-second propagation characteristic of Supabase's WebSocket-based realtime engine described in Section 2.14. The Dijkstra routing computation itself completed in low single-digit milliseconds at the graph sizes used in testing (tens of nodes), confirming the O((V + E) log V) complexity analysis in Section 2.6 translates into negligible real-world latency at this scale — the dominant cost in the end-to-end SOS workflow is network round-trip time to the AI Gateway for triage inference, not the routing computation. Dashboard load times, measured from authentication to first meaningful paint, remained consistently fast owing to Vite's optimised production bundling and the use of narrowly-scoped realtime subscriptions rather than broad table-wide listeners.

3.21 Achieved Objectives

Measured against the objectives set out in Section 1.5, the implemented system delivers a working realtime platform connecting all five target roles on a shared data model; a functioning AI Triage module producing structured, confidence-scored clinical assessments; a correct, verifiable Dijkstra routing implementation with an interactive visualiser that lets the algorithm's behaviour be inspected directly rather than trusted blindly; a normalised, RLS-secured PostgreSQL schema covering all fourteen planned tables; dual-path authentication with email verification and Google OAuth; a realtime Live Emergency Map; and the full set of supporting modules — medical records, appointments, emergency contacts, notifications, global search and PDF report generation. All nine objectives listed in Section 1.5 were achieved within the scope defined in Section 1.6.

3.22 Future Scope of the Project

While the current implementation satisfies its stated objectives, several extensions would meaningfully increase its real-world readiness:

- IoT Device Integration — ingesting live vitals directly from wearable sensors (heart-rate straps, pulse oximeters) into the AI Triage pipeline, removing manual vitals entry during an emergency.
- Wearable Device Alerts — allowing a smartwatch to trigger SOS directly on detecting a fall or abnormal heart rhythm, without requiring the patient to reach for a phone.
- AI Disease Prediction — extending the AI Gateway to flag longer-term risk patterns from historical medical_records entries, rather than only assessing the current acute episode.
- Predictive Ambulance Dispatch — pre-positioning ambulances in anticipation of demand using historical sos_requests patterns by time of day and location, rather than dispatching reactively.
- Multi-language Support — localising the interface and, where feasible, the AI Triage symptom intake, to serve non-English-speaking patients directly.
- Cloud Scaling — load-testing and, where necessary, sharding the realtime and database layers for metropolitan-scale concurrent usage well beyond the graph sizes evaluated in Section 3.20.
- Telemedicine Integration — adding live video consultation directly from the Patient Portal for cases the AI Triage module classifies as non-critical, reducing unnecessary in-person visits.
- Advanced Analytics — city-wide dashboards for public-health authorities aggregating anonymised trends across all participating hospitals.
- Smart City Integration — exchanging live traffic-signal and road-closure data with municipal traffic-management systems to improve the accuracy of the Dijkstra edge weights described in Section 2.3 beyond the current traffic-tier approximation.

Conclusion

MediRoute AI demonstrates that the core weaknesses of manually coordinated emergency healthcare response — no single point of digital contact, no automated severity assessment, sub-optimal hospital selection, and fragmented records — can be addressed by combining three well-understood building blocks inside one realtime platform: a structured AI triage step, a classical shortest-path routing algorithm, and a shared, row-level-secured realtime database connecting every stakeholder to the same live picture of an emergency. The project's five role-based dashboards, its fourteen-table PostgreSQL schema, and its working Dijkstra implementation together show that the integration gap identified in Section 1.4 — between clinical assessment and logistics — can be closed with currently available, production-grade tools rather than bespoke infrastructure.

Beyond the working software itself, the project served as a substantial exercise in full-stack engineering practice: translating a real-world coordination problem into a normalised relational schema; implementing and verifying a graph algorithm well enough to expose its internal state through an interactive visualiser rather than treating it as a black box; designing authentication and authorization so that access control is enforced at the database layer rather than trusted to client-side routing; and building a realtime system where every 'live' indicator in the interface is backed by an actual WebSocket subscription rather than a polling loop dressed up to look instantaneous. Each of these was a deliberate design decision made and justified during the course of the project, not a default inherited from a starting template.

The system's benefits, evaluated in Chapter 3, are concrete: sub-second realtime propagation of emergency status between patient and hospital, millisecond-scale routing computation, and a structured audit trail — triage reports, SOS timelines and notification history — that a purely manual, phone-call-based process cannot provide. At the same time, the Future Scope discussion in Section 3.22 is an honest account of what remains before the system could be operated at genuine city scale: live IoT vitals ingestion, predictive dispatch, and integration with municipal traffic data are all identified as the next logical steps rather than claimed as already solved. Taken together, MediRoute AI meets its stated objectives as an academic project while also outlining a credible path toward a deployable emergency-response platform.

References

[1] E. W. Dijkstra, "A note on two problems in connexion with graphs," Numerische Mathematik, vol. 1, no. 1, pp. 269–271, 1959.

[2] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in Advances in Neural Information Processing Systems (NeurIPS), 2017.

[3] Meta Platforms, Inc., "React – The library for web and native user interfaces," React documentation. [Online]. Available: https://react.dev

[4] Microsoft Corp., "TypeScript documentation," TypeScript. [Online]. Available: https://www.typescriptlang.org/docs/

[5] Vite contributors, "Vite – Next generation frontend tooling," Vite documentation. [Online]. Available: https://vitejs.dev

[6] Tailwind Labs Inc., "Tailwind CSS documentation," Tailwind CSS. [Online]. Available: https://tailwindcss.com/docs

[7] Supabase Inc., "Supabase documentation," Supabase. [Online]. Available: https://supabase.com/docs

[8] The PostgreSQL Global Development Group, "PostgreSQL 16 documentation," PostgreSQL. [Online]. Available: https://www.postgresql.org/docs/

[9] Google LLC, "Sign in with Google – Identity documentation," Google for Developers. [Online]. Available: https://developers.google.com/identity

[10] D. Hardt, Ed., "The OAuth 2.0 authorization framework," IETF RFC 6749, Oct. 2012. [Online]. Available: https://www.rfc-editor.org/rfc/rfc6749

[11] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," IETF RFC 7519, May 2015. [Online]. Available: https://www.rfc-editor.org/rfc/rfc7519

[12] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th ed. Cambridge, MA: MIT Press, 2022, ch. 22 (Single-Source Shortest Paths).